

گروه D

محمدامین حیدری 401170553

سینا محمدی 401110064

امیرحسین قاسمی پور 401106341

۱. هدف آزمایش

هدف این آزمایش، طراحی و پیاده‌سازی یک واحد محاسبه و منطق (ALU) برای اعداد مختلط با Verilog و سپس قرار دادن آن در یک کامپیوتر پایه با حافظه ۳۲ کلمه‌ای به صورت پایلایین است. ماشین فرضی پیاده‌سازی شده، دستورات را از حافظه دستورالعمل خوانده و عملیات مختلط (جمع، تفریق، ضرب) را روی اعداد مختلط ذخیره شده در حافظه داده اجرا می‌کند.

این آزمایش از سه بخش اصلی تشکیل شده است:

- پیمانانه جمع و تفریق اعداد مختلط
- پیمانانه ضرب اعداد مختلط
- واحد پایلایین که دستورات را از حافظه خوانده و به صورت پایلایین اجرا می‌کند

۲. محدودیت اصلی طراحی

مطابق با صورت آزمایش، در این طراحی نبایستی از واحدهای جمع‌کننده یا ضرب‌کننده بیش از یک بار استفاده شود. این بدان معناست که در کل طراحی تنها یک ضرب‌کننده حقیقی (mul) و یک جمع/تفریق‌کننده حقیقی (addsub) قابل استفاده است.

این محدودیت اساسی‌ترین چالش طراحی است، زیرا ضرب دو عدد مختلط نیاز به چهار ضرب حقیقی و دو جمع/تفریق دارد. برای رعایت این محدودیت، یک ALU چندسیکلی طراحی شده است که با استفاده از یک ماشین حالت (FSM)، واحدهای مشترک را در سیکل‌های مختلف بین عملیات مختلف زمان-تقسیم می‌کند.

۳. ساختار کلی پروژه

پروژه از فایل‌های زیر تشکیل شده است:

- macros.v — تعاریف ماکروها (عرض کلمه، دسترسی به بخش حقیقی و موهومی)
- addsub.v — یک جمع/تفریق‌کننده حقیقی ۸ بیتی (تنها یک نمونه)
- mul.v — یک ضرب‌کننده حقیقی ۸ بیتی (تنها یک نمونه)
- alu.v — چندسیکلی که واحدهای بالا را زمان-تقسیم می‌کند ALU
- inst_fetch.v — (PC) حافظه دستورالعمل و شمارنده برنامه
- memory.v — حافظه داده با دو پورت خواندن و یک پورت نوشتن

- pipeline.v — مازول سطح بالاي پايپلاين
- *_TB.v — تست‌بنج‌هاي چهارگانه
- data/inst_mem.txt و data/initial_mem.txt — مقادير اوليه حافظه

۴. تشریح فایل‌ها و کد

۴-۱. فایل macros.v

این فایل تعاریف اصلي پروژه را در بر دارد. عرض کلمه (WL) برابر ۸ بیت تعریف شده و یک عدد مختلط به صورت کلمه ۱۶ بیتی ذخیره می‌شود که ۸ بیت بالایی بخش حقیقی و ۸ بیت پایین بخش موهومی است.

```
`define WL 8
`define complex [2*`WL-1:0]
`define Re(c) c[2*`WL-1:`WL]
`define Im(c) c[`WL-1:0]
`define sRe(c) $signed(`Re(c))
`define sIm(c) $signed(`Im(c))
```

۴-۲. فایل addsub.v

این فقط شامل يك جمع‌کننده/تفریق‌کننده حقیقی ۸ بیتی است. ALU این پیمانۀ را در سیکل‌هاي مختلف هم برای جمع/تفریق مختلط (یک‌بار برای بخش حقیقی و یک‌بار برای موهومی) و هم برای مراحل پایانی ضرب مختلط استفاده می‌کند.

```
module addsub (
    input signed [`WL-1:0] x,
    input signed [`WL-1:0] y,
    input op,
    output signed [`WL-1:0] s
);
    assign s = op ? (x - y) : (x + y);
endmodule
```

۴-۳. فایل mul.v

این فقط يك ضرب‌کننده حقیقی ۸ بیتی است. خروجی به ۸ بیت کوتاه (truncate) می‌شود تا با عرض ذخیره‌سازی اعداد مختلط سازگار باشد. برای محاسبه ضرب مختلط، ALU این واحد را در چهار سیکل پشت سر هم با ورودی‌هاي متفاوت استفاده می‌کند.

```
module mul (
    input signed [`WL-1:0] x,
    input signed [`WL-1:0] y,
    output signed [`WL-1:0] p
);
```

```
assign p = x * y;
endmodule
```

۴-۴. فایل alu.v (هسته اصلي طراحي)

این قلب طراحي است. ALU به صورت چندسیکلي طراحي شده و دقیقاً يك نمونه از mul و يك نمونه از addsub را با كمك يك ماشين حالت ۸ حالتۀ زمان-تقسيم مي‌كند.

كدگذاري عمليات (op):

op	عمل	تعداد سيكل
00	جمع مختلط	۲
01	تفريق مختلط	۲
1x	ضرب مختلط	۵

حالت‌هاي ماشين حالت:

```
localparam IDLE = 4'd0,
            AS1  = 4'd1, // Re(result)
            AS2  = 4'd2, // Im(result)
            M1   = 4'd3, // mul: Re*Re
            M2   = 4'd4, // mul: Im*Im
            M3   = 4'd5, // mul: Re*Im
            M4   = 4'd6, // mul: Im*Re
            M5   = 4'd7; //
```

بخش زیر منطق ترکیبی ورودی‌هاي mul و addsub را بر اساس حالت فعلی تنظیم می‌کند. در حالت M3 هم‌زمان ضرب‌کننده مشغول محاسبه ad است و جمع‌کننده در حال محاسبه ac - bd برای بخش حقیقی خروجی ضرب است:

```
always @(*) begin
    mul_x = 0; mul_y = 0;
    as_x  = 0; as_y  = 0; as_op = 0;
    case (state)
        AS1: begin as_x = `sRe(a); as_y = `sRe(b); as_op = sub; end
        AS2: begin as_x = `sIm(a); as_y = `sIm(b); as_op = sub; end
        M1 : begin mul_x = `sRe(a); mul_y = `sRe(b); end
        M2 : begin mul_x = `sIm(a); mul_y = `sIm(b); end
        M3 : begin
            mul_x = `sRe(a); mul_y = `sIm(b);
            as_x  = t_ac;   as_y  = t_bd;   as_op = 1'b1;
        end
        M4 : begin mul_x = `sIm(a); mul_y = `sRe(b); end
        M5 : begin as_x = t_ad; as_y = t_bc; as_op = 1'b0; end
    endcase
end
```

```
endcase
end
```

جدول زمان‌بندی ضرب مختلط

این جدول نشان می‌دهد که در طول ۵ سیکل ضرب مختلط، ضرب‌کننده و جمع‌کننده اشتراکی چه عملیاتی انجام می‌دهند. هرگز در یک سیکل بیش از یک ضرب یا بیش از یک جمع/تفریق رخ نمی‌دهد.

سیکل	ضرب‌کننده	جمع/تفریق‌کننده	ثبت شدن در لبه بعد
M1	$\text{Re}(a) \times \text{Re}(b)$	بیکار	—
M2	$\text{Im}(a) \times \text{Im}(b)$	بیکار	$t_{ac} \leftarrow ac$
M3	$\text{Re}(a) \times \text{Im}(b)$	$ac - bd$	$t_{bd} \leftarrow bd$
M4	$\text{Im}(a) \times \text{Re}(b)$	بیکار	$t_{ad} \leftarrow ad$ ، $\text{Re}(\text{result}) \leftarrow ac - bd$
M5	بیکار	$ad + bc$	$t_{bc} \leftarrow bc$
بعد	—	—	$\text{Im}(\text{result}) \leftarrow ad + bc$ ، $\text{done} = 1$

۴-۵. فایل inst_fetch.v

حافظه دستورالعمل ۳۲ کلمه‌ای همراه با شمارنده برنامه (PC). هر دستورالعمل ۱۷ بیت است و شامل ۲ بیت ۵، op، بیت آدرس مقصد، و دو فیلد ۵ بیتی برای آدرس‌های خواندن است. ورودی stall کنترلر را قادر می‌سازد در زمان‌هایی که ALU مشغول است، PC را منجمد نگه دارد.

```
reg [16:0]      mem [DEPTH-1:0];
reg [A_LEN-1:0] pc;

assign {op, waddr, raddr1, raddr2} = mem[pc];

always @(posedge clk or negedge rstN) begin
    if (!rstN)      pc <= 0;
    else if (!stall) pc <= pc + 1'b1;
end
```

۴-۶. فایل memory.v

حافظه داده با دو پورت خواندن ناهمزمان (ترکیبی) و یک پورت نوشتن همزمان (با لبه ساعت). با این ساختار، در یک سیکل می‌توان هم دو عملوند جدید را خواند و هم نتیجه‌ی دستور قبلی را نوشت.

```
reg `complex mem [DEPTH-1:0];

assign rdata1 = mem[raddr1];
assign rdata2 = mem[raddr2];

always @(posedge clk) begin
```

```
if (wen) mem[waddr] <= wdata;
end
```

۴-۷. فایل pipeline.v (واحد سطح بالا)

این پیمانۀ IF، MEM و EX را به هم متصل می‌کند. چون ALU چندسیکلی است، نمی‌توان به سادگی یک دستور در هر سیکل صادر کرد. به جای آن از یک دست‌دهی (handshake) استفاده شده است: سیگنال ready_to_issue نشان می‌دهد که آیا می‌توان دستور جدیدی را به ALU سپرد یا نه. وقتی ALU سیگنال done را ارسال می‌کند، در همان لبه ساعت دستور بعدی گرفته می‌شود و در نتیجه فاصله بین دو دستور پشت سر هم تنها یک سیکل سربار اضافی دارد.

```
wire ready_to_issue = !busy || alu_done;
wire stall          = busy && !alu_done;
wire wb_en          = alu_done;

always @(posedge clk or negedge rstN) begin
    if (!rstN) begin
        busy      <= 1'b0;
        alu_start <= 1'b0;
    end else begin
        alu_start <= 1'b0;
        if (ready_to_issue) begin
            cur_op    <= if_op;
            cur_waddr <= if_waddr;
            cur_a     <= rdata1;
            cur_b     <= rdata2;
            alu_start <= 1'b1;
            busy      <= 1'b1;
        end
    end
end
```

۵. مراحل اجرا در Quartus

۵-۱. سنتز و کامپایل

از منوی **Processing** گزینه **Start Compilation** را اجرا کنید (یا Ctrl+L). بعد از اتمام، گزارش منابع از مسیر زیر قابل مشاهده است:

Compilation Report → Analysis & Synthesis → Resource Usage Summary

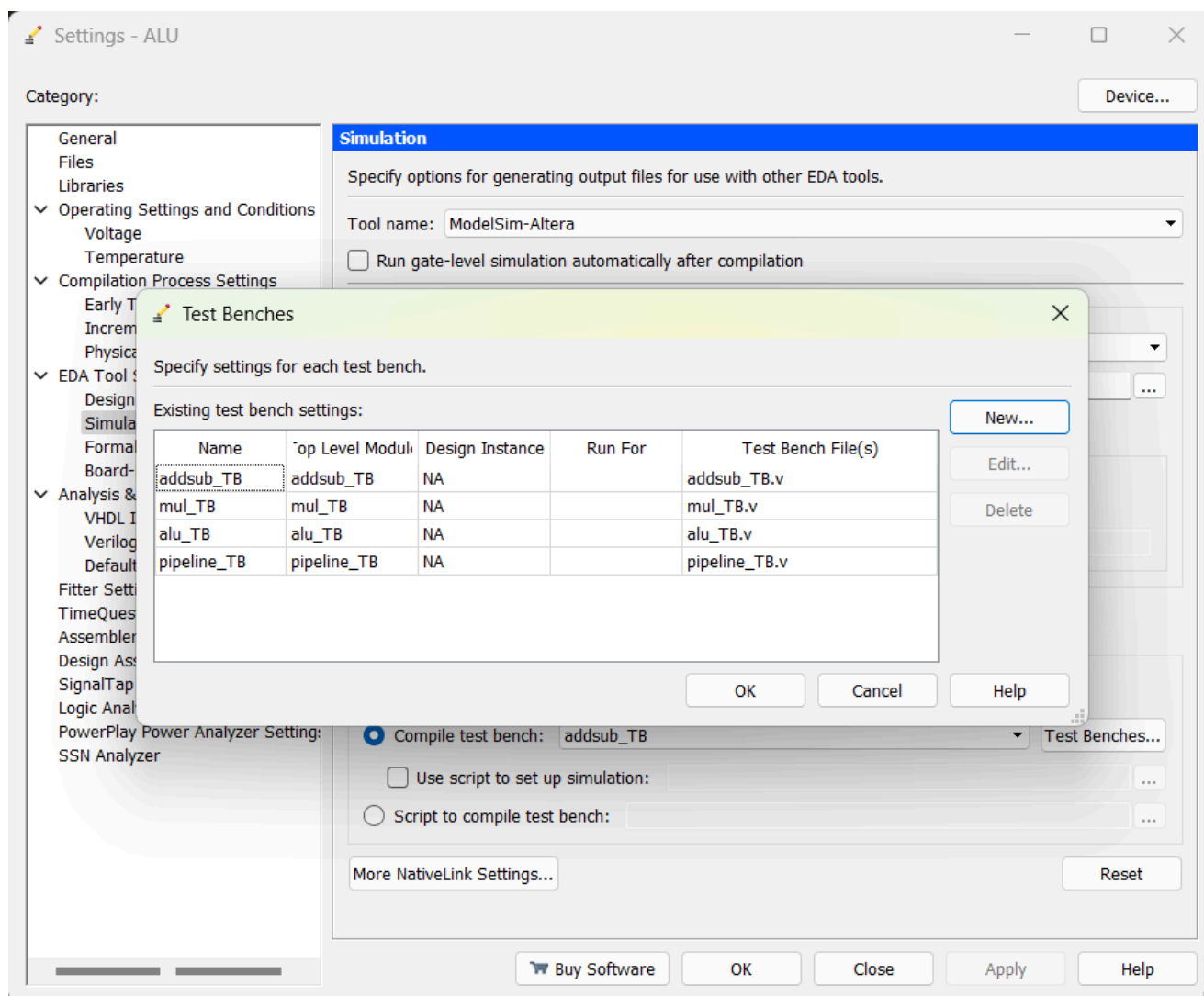
نکته: ماژول pipeline هیچ پورت خروجی ندارد، بنابراین سنتزکننده هشدار می‌دهد که بسیاری از سیگنال‌ها بهینه‌سازی شده‌اند. برای گرفتن گزارش منابع واقعی از ALU، می‌توانید به طور موقت alu را به عنوان ماژول سطح بالا تنظیم کرده و دوباره کامپایل کنید.

Flow Summary	
Flow Status	Successful - Thu Jun 04 20:44:51 2026
Quartus II 64-Bit Version	13.0.1 Build 232 06/12/2013 SP 1 SJ Web Edition
Revision Name	ALU
Top-level Entity Name	pipeline
Family	Cyclone II
Device	EP2C5AF256A7
Timing Models	Final
Total logic elements	0 / 4,608 (0 %)
Total combinational functions	0 / 4,608 (0 %)
Dedicated logic registers	0 / 4,608 (0 %)
Total registers	0
Total pins	2 / 158 (1 %)
Total virtual pins	0
Total memory bits	0 / 119,808 (0 %)
Embedded Multiplier 9-bit elements	0 / 26 (0 %)
Total PLLs	0 / 2 (0 %)

۵-۲. تنظیم تست‌بنچ‌ها در NativeLink

از مسیر Assignments → Settings → EDA Tool Settings → Simulation ...NativeLink settings → Compile test bench → Test Benches کلیک کنید. با دکمه **New** چهار تست‌بنچ را اضافه کنید:

Test bench name	Top level module	File
addsub_TB	addsub_TB	addsub_TB.v
mul_TB	mul_TB	mul_TB.v
alu_TB	alu_TB	alu_TB.v
pipeline_TB	pipeline_TB	pipeline_TB.v



۶. شبیه‌سازی در ModelSim

۶-۱. اجرای شبیه‌سازی

برای اجرای هر تست‌بنچ، در صفحه تنظیمات شبیه‌سازی، آن را از منوی کرکره‌ای **Compile test bench** انتخاب کنید و سپس از منوی **Tools** گزینه **Run Simulation Tool** → **RTL Simulation** را اجرا کنید.

۶-۲. مدیریت پوشه data

تست‌بنچ pipeline_TB از readmemb\$ برای بارگذاری حافظه دستورالعمل و حافظه داده استفاده می‌کند. ModelSim از مسیر /simulation/modelsim اجرا می‌شود، بنابراین باید پوشه data در همین مسیر قرار گیرد:

```
project_folder/
simulation/
modelsim/
data/
inst_mem.txt
initial_mem.txt
```

۷. نتایج شبیه‌سازی

۷-۱. تست‌بنچ addsub

این تست‌بنچ تنها رفتار يك جمع/تفریق‌کننده ۸ بیتی حقيقي را بررسی می‌کند. خروجی:

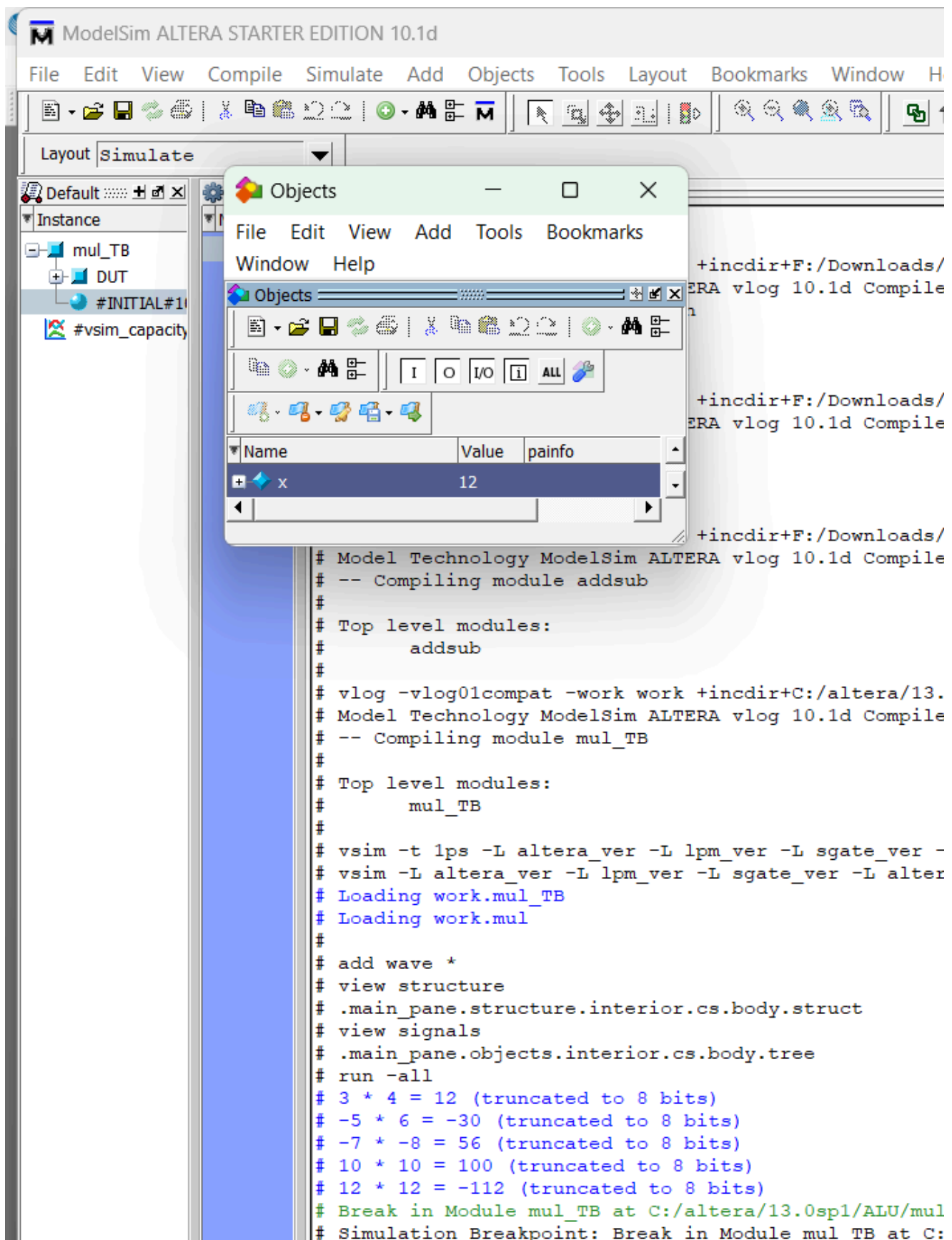
```
10 + 5 = 15
10 - 5 = 5
-20 + 30 = 10
50 - -50 = 100
```

```
# .main_pane.objects.inter
# run -all
# 10 + 5 = 15
# 10 - 5 = 5
# -20 + 30 = 10
# 50 - -50 = 100
# Break in Module addsub_1
# Simulation Breakpoint: 5
```

۷-۲. تست‌بنچ mul

این تست‌بنچ يك ضرب‌کننده حقيقي را تست می‌کند. در حالت آخر، $12 \times 12 = 144$ از محدوده ۸ بیت علامت‌دار خارج می‌شود و به -112 می‌رسد که پدیده truncation را نشان می‌دهد.

```
3 * 4 = 12
-5 * 6 = -30
-7 * -8 = 56
10 * 10 = 100
12 * 12 = -112 (overflow)
```

۷-۳. تست‌بنج alu

این تست‌بنج مهم‌ترین بخش است: ALU را به طور کامل با عملیات‌های جمع، تفریق، و ضرب مختلط آزمایش می‌کند. نتیجه مورد انتظار:

```
op=00 (3, 4) + (1, 2) = (4, 6)    ← complex add
op=01 (3, 4) - (1, 2) = (2, 2)    ← complex sub
```

```
op=10 (3, 4) * (1, 2) = (-5, 10) ← complex mul
op=00 (-7, 5) + (7, -5) = (0, 0)
```

محاسبه ضرب: $(3 + 4i)(1 + 2i) = (-5 + 10i)$. این نتیجه با استفاده از فقط يك ضرب‌کننده و يك جمع‌کننده در ۵ سیکل به دست آمده است.

```
# add wave *
# view structure
# .main_pane.structure.interior.cs.body.struct
# view signals
# .main_pane.objects.interior.cs.body.tree
# run -all
# op=00 (3, 4) ? (1, 2) = (4, 6)
# op=01 (3, 4) ? (1, 2) = (2, 2)
# op=10 (3, 4) ? (1, 2) = (-5, 10)
# op=00 (-7, 5) ? (7, -5) = (0, 0)
# Break in Module alu_TB at C:/altera/13.0sp1/ALU/alu_TB.v
# Simulation Breakpoint: Break in Module alu_TB at C:/alter
# MACRO ./ALU_run_msim_rtl_verilog.do PAUSED at line 23

VSIM(paused)>
```

۷-۴. تست‌بنچ pipeline

این تست‌بنچ کامل‌ترین بخش شبیه‌سازی است. يك برنامه شامل ۴ دستور اجرا می‌شود:

- $\text{mem}[2] \leftarrow \text{mem}[0] + \text{mem}[1] = (3,4) + (1,2) = (4, 6)$
- $\text{mem}[3] \leftarrow \text{mem}[0] - \text{mem}[1] = (3,4) - (1,2) = (2, 2)$
- $\text{mem}[4] \leftarrow \text{mem}[0] \times \text{mem}[1] = (3,4) \times (1,2) = (-5, 10)$
- $\text{mem}[5] \leftarrow \text{mem}[2] + \text{mem}[3] = (4,6) + (2,2) = (6, 8) \leftarrow$ وابستگی به نتایج قبلي

خروجي ModelSim:

```
# run -all
#120 WB mem[2] <= (4, 6) (op=00, a=(3,4), b=(1,2))
#200 WB mem[3] <= (2, 2) (op=01, a=(3,4), b=(1,2))
#340 WB mem[4] <= (-5, 10) (op=10, a=(3,4), b=(1,2))
#420 WB mem[5] <= (6, 8) (op=00, a=(4,6), b=(2,2))
#500 WB mem[31] <= (6, 8) (op=00, a=(3,4), b=(3,4))
#580 WB mem[31] <= (6, 8) (op=00, a=(3,4), b=(3,4))
#660 WB mem[31] <= (6, 8) (op=00, a=(3,4), b=(3,4))
#740 WB mem[31] <= (6, 8) (op=00, a=(3,4), b=(3,4))
#820 WB mem[31] <= (6, 8) (op=00, a=(3,4), b=(3,4))
# ---- final memory ----
# mem[0] = (3, 4)
# mem[1] = (1, 2)
# mem[2] = (4, 6)
# mem[3] = (2, 2)
# mem[4] = (-5, 10)
# mem[5] = (6, 8)
# mem[6] = (0, 0)
# mem[7] = (0, 0)
# Break in Module pipeline_TB at C:/altera/13.0sp1/ALU/pipeline
# Simulation Breakpoint: Break in Module pipeline_TB at C:/alte
```

دستور چهارم: عملوندهای آن مقادیری هستند که در دو دستور قبلي نوشته شده‌اند. اینکه نتیجه آن (۸، ۶) درست به دست می‌آید، تأیید می‌کند که منطق stall و نوشتن بازگشت (writeback) به درستی وابستگی داده را مدیریت کرده است.

